VALORANT ESPORTS TOURNAMENT ECOSYSTEM

DBMS Project

B.Tech Computer Science & Engineering

Academic Year 2025 - 26

Group Members

Abhay Kumar · 2410030695 · **Group Leader**

Dushyant Kumar · 2410030677 · Member

Vaishnavi Sharma · 2410030681 · Member

Deepak Kumar · 2410030660 · Member

1. Initial Database Schema

The schema designed in Task 2 consists of six interrelated tables. Each table is listed below with its column set and key constraints, exactly as submitted.

Table: Tournaments

Column	Type	Key / Constraint
tournament_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
region	VARCHAR(50)	NOT NULL
format	VARCHAR(20)	NOT NULL
start_date	DATE	NOT NULL
end_date	DATE	NULL
prize_pool	DECIMAL(12,2)	—

Table: Teams

Column	Type	Key / Constraint
team_id	INT	PRIMARY KEY, AUTO_INCREMENT
team_name	VARCHAR(100)	NOT NULL, UNIQUE
region	VARCHAR(50)	NOT NULL
founded_date	DATE	—

Table: Players

Column	Type	Key / Constraint
player_id	INT	PRIMARY KEY, AUTO_INCREMENT
ign	VARCHAR(50)	NOT NULL, UNIQUE
real_name	VARCHAR(100)	—
primary_role	VARCHAR(30)	NOT NULL
country	VARCHAR(50)	—
team_id	INT	FK → Teams(team_id)

Table: Matches

Column	Type	Key / Constraint
match_id	INT	PRIMARY KEY, AUTO_INCREMENT

Column	Type	Key / Constraint
tournament_id	INT	FK → Tournaments(tournament_id)
team_a_id	INT	FK → Teams(team_id)
team_b_id	INT	FK → Teams(team_id)
match_format	VARCHAR(5)	NOT NULL
match_date	DATETIME	NOT NULL
winner_id	INT	FK → Teams(team_id), NULL if ongoing

Table: Match_Maps

Column	Type	Key / Constraint
map_result_id	INT	PRIMARY KEY, AUTO_INCREMENT
match_id	INT	FK → Matches(match_id)
map_number	INT	NOT NULL
map_name	VARCHAR(50)	NOT NULL
team_a_score	INT	NOT NULL
team_b_score	INT	NOT NULL
map_winner_id	INT	FK → Teams(team_id)

Table: Player_Stats

Column	Type	Key / Constraint
stat_id	INT	PRIMARY KEY, AUTO_INCREMENT
match_id	INT	FK → Matches(match_id)
map_result_id	INT	FK → Match_Maps(map_result_id)
player_id	INT	FK → Players(player_id)
agent_played	VARCHAR(50)	NOT NULL
kills	INT	DEFAULT 0
deaths	INT	DEFAULT 0
assists	INT	DEFAULT 0
acs	DECIMAL(6,2)	DEFAULT 0
headshot_pct	DECIMAL(5,2)	DEFAULT 0

2. Functional Dependencies

A functional dependency X → Y means that the value of X uniquely determines Y. Below are all non-trivial FDs for each table, with their type noted.

FDs: Tournaments

Determinant	Determined Attribute(s)	Type
tournament_id	name, region, format, start_date, end_date, prize_pool	Full / Trivial PK

FDs: Teams

Determinant	Determined Attribute(s)	Type
team_id	team_name, region, founded_date	Full / Trivial PK
team_name	team_id, region, founded_date	Candidate Key (UNIQUE)

FDs: Players

Determinant	Determined Attribute(s)	Type
player_id	ign, real_name, primary_role, country, team_id	Full / Trivial PK
ign	player_id, real_name, primary_role, country, team_id	Candidate Key (UNIQUE)

FDs: Matches

Determinant	Determined Attribute(s)	Type
match_id	tournament_id, team_a_id, team_b_id, match_format, match_date, winner_id	Full / Trivial PK

FDs: Match_Maps

Determinant	Determined Attribute(s)	Type
map_result_id	match_id, map_number, map_name, team_a_score, team_b_score, map_winner_id	Full / Trivial PK
match_id, map_number	map_result_id, map_name, team_a_score, team_b_score, map_winner_id	Composite Candidate Key

FDs: Player_Stats

Determinant	Determined Attribute(s)	Type
stat_id	match_id, map_result_id, player_id, agent_played, kills, deaths, assists, acs, headshot_pct	Surrogate PK

Determinant	Determined Attribute(s)	Type
{map_result_id, player_id}	agent_played, kills, deaths, assists, acs, headshot_pct	Natural Candidate Key (full)
map_result_id	match_id	PARTIAL — see Section 4

Note on Player_Stats: *The natural candidate key is* ***{map_result_id, player_id}****. The column* ***match_id*** *is functionally determined by map_result_id alone (because Match_Maps already records which match each map belongs to). This means match_id has a partial dependency on the candidate key — it only needs one half of the key to be identified, not both.*

3. First Normal Form (1NF)

Definition

A table is in 1NF when every column holds atomic (indivisible) values, there are no repeating groups or multi-valued columns, and every row is uniquely identified by a primary key.

Verification — All Six Tables

Table	Atomic Values?	No Repeating Groups?	Primary Key?	In 1NF?
Tournaments	Yes	Yes	tournament_id (PK)	Yes
Teams	Yes	Yes	team_id (PK)	Yes
Players	Yes	Yes	player_id (PK)	Yes
Matches	Yes	Yes	match_id (PK)	Yes
Match_Maps	Yes	Yes	map_result_id (PK)	Yes
Player_Stats	Yes	Yes	stat_id (PK)	Yes

Justification

- Every column stores a single value. player_id is a single integer, not a comma-separated list. agent_played is a single agent name per row.
- No column hides multiple values inside one field. Map scores are stored in two separate columns (team_a_score, team_b_score) rather than combined strings.
- Per-map statistics each get their own row in Player_Stats instead of being stacked into arrays or nested structures.
- Each table has a surrogate primary key that uniquely identifies every row.

Conclusion: *All six tables satisfy 1NF as originally designed. No structural changes are required at this step.*

4. Second Normal Form (2NF)

Definition

A table is in 2NF when it is already in 1NF and every non-key attribute is fully functionally dependent on the entire primary key — no partial dependencies on a subset of a composite key.

Checking Tables with Surrogate (Single-Column) PKs

Tournaments, Teams, Players, Matches, and Match_Maps each use a single-column surrogate key. A partial dependency requires a composite key, so these five tables cannot have partial dependencies by definition. They satisfy 2NF automatically.

The Problem: Player_Stats

Player_Stats is where the issue sits. The surrogate key is stat_id, which alone does not reveal the structural problem. Looking at the **natural candidate key** exposes it.

Natural Candidate Key Analysis

A player's performance on a specific map in a specific match is uniquely identified by **{map_result_id, player_id}**. This is the meaningful key for this table.

Now examine every non-key column against this composite candidate key:

Non-Key Column	Depends On	Full or Partial?
agent_played	{map_result_id, player_id}	Full — agent choice is per player per map
kills	{map_result_id, player_id}	Full — stat is per player per map
deaths	{map_result_id, player_id}	Full — stat is per player per map
assists	{map_result_id, player_id}	Full — stat is per player per map
acs	{map_result_id, player_id}	Full — stat is per player per map
headshot_pct	{map_result_id, player_id}	Full — stat is per player per map
match_id	map_result_id only	PARTIAL — map_result_id → match_id via Match_Maps

Why This Is a Real Problem

match_id is already recorded in Match_Maps. Given any map_result_id, the corresponding match_id is immediately recoverable:

```
SELECT match_id FROM Match_Maps WHERE map_result_id = <value>;
```

Storing match_id directly in Player_Stats introduces redundancy. If a match record were reassigned to a different tournament (changing its match_id chain), the value in Player_Stats would have to be updated separately — a classic update anomaly.

Fix — Decompose Player_Stats to Remove the Partial Dependency

Remove match_id from Player_Stats. All performance data is still reachable through a JOIN via map_result_id:

```
Player_Stats JOIN Match_Maps USING (map_result_id) → gives match_id Match_Maps JOIN
Matches USING (match_id) → gives tournament_id, teams, date
```

Player_Stats — Before (1NF, not 2NF)

Column	Notes
`stat_id`	Surrogate PK
`match_id`	**REDUNDANT — partial dependency on map_result_id**
`map_result_id`	FK → Match_Maps
`player_id`	FK → Players
`agent_played`	
`kills / deaths / assists`	
`acs / headshot_pct`	

Player_Stats — After (2NF)

Column	Notes
`stat_id`	Surrogate PK
`map_result_id`	FK → Match_Maps (match_id now derived from here)
`player_id`	FK → Players
`agent_played`	
`kills`	
`deaths`	
`assists`	
`acs`	
`headshot_pct`	

Justification: *Removing match_id eliminates the partial dependency. Every remaining non-key column (agent_played, kills, deaths, assists, acs, headshot_pct) depends on the full natural key {map_result_id, player_id}. No information is lost — match_id is always derivable via Match_Maps. The other five tables were already in 2NF and remain unchanged.*

5. Third Normal Form (3NF)

Definition

A table is in 3NF when it is in 2NF and no non-key attribute is transitively dependent on the primary key through another non-key attribute. Formally: for every FD X → Y in the table, either X is a superkey, or Y is a prime attribute (member of some candidate key).

Checking All Six Tables for Transitive Dependencies

Table	Non-Key → Non-Key Path?	Transitive Dep?	3NF?
Tournaments	No non-key column determines another (name/region/format are independent facts about the tournament)	None	**Yes**
Teams	team_name and region are both facts about the team; neither determines the other	None	**Yes**
Players	primary_role and country are facts about the player; team_id is an FK (not a non-key fact); no chain exists	None	**Yes**
Matches	match_format and match_date are facts about the match; team FKs are just references; no non-key column determines another	None	**Yes**
Match_Maps	map_name, scores, and map_winner_id are facts about the map instance; map_winner_id does not determine map_name or scores	None	**Yes**
Player_Stats (after 2NF fix)	agent_played does not determine kills/acs/headshot_pct — two players can play the same agent and have completely different stats	None	**Yes**

Special Note — Players.team_id

One column worth discussing separately is team_id in the Players table. It might look like a transitive path: player_id → team_id → team_name. However, this is **not a transitive dependency** within the Players table because team_id is a foreign key — it is a reference to a separate entity (Teams), not a non-key attribute that itself has further dependencies inside Players. The rule applies to dependencies that exist within the same table's non-key columns. The Teams table handles team_id → team_name, and it does so correctly with team_id as the primary key.

Conclusion: *After the 2NF fix (removing match_id from Player_Stats), all six tables are free of transitive dependencies. The entire schema is in 3NF.*

6. Final Set of Normalized Tables (3NF)

Five of the six tables are carried over unchanged. Only Player_Stats is modified: match_id is removed. The final schema is listed below.

Tournaments — unchanged

```
tournament_id (PK), name, region, format, start_date, end_date, prize_pool
```

Teams — unchanged

```
team_id (PK), team_name (UNIQUE), region, founded_date
```

Players — unchanged

```
player_id (PK), ign (UNIQUE), real_name, primary_role, country, team_id (FK)
```

Matches — unchanged

```
match_id (PK), tournament_id (FK), team_a_id (FK), team_b_id (FK), match_format,
match_date, winner_id (FK)
```

Match_Maps — unchanged

```
map_result_id (PK), match_id (FK), map_number, map_name, team_a_score, team_b_score,
map_winner_id (FK)
```

Player_Stats — match_id REMOVED (2NF fix)

```
stat_id (PK), map_result_id (FK), player_id (FK), agent_played, kills, deaths, assists,
acs, headshot_pct
```

Summary of Changes

- **match_id removed from Player_Stats.** This was the only structural change needed. The column was a partial dependency on the natural candidate key {map_result_id, player_id} because map_result_id already encodes the match via the Match_Maps table.
- All five other tables were already in 3NF and required no modification.
- No information is lost. match_id is always reachable: Player_Stats → Match_Maps (via map_result_id) → Matches (via match_id).

The schema satisfies all three normal forms after this single targeted fix. The design avoids update anomalies, insertion anomalies, and deletion anomalies across all six tables.